



Kubernetes and AI To Protect Our Forests

Cloud Native Infrastructure for Wildfire Prevention

Andrea Giardini
Cloud Native Consultant at Overstory
contact@andreagiardini.com



January 2025 Southern California wildfires

29 deaths

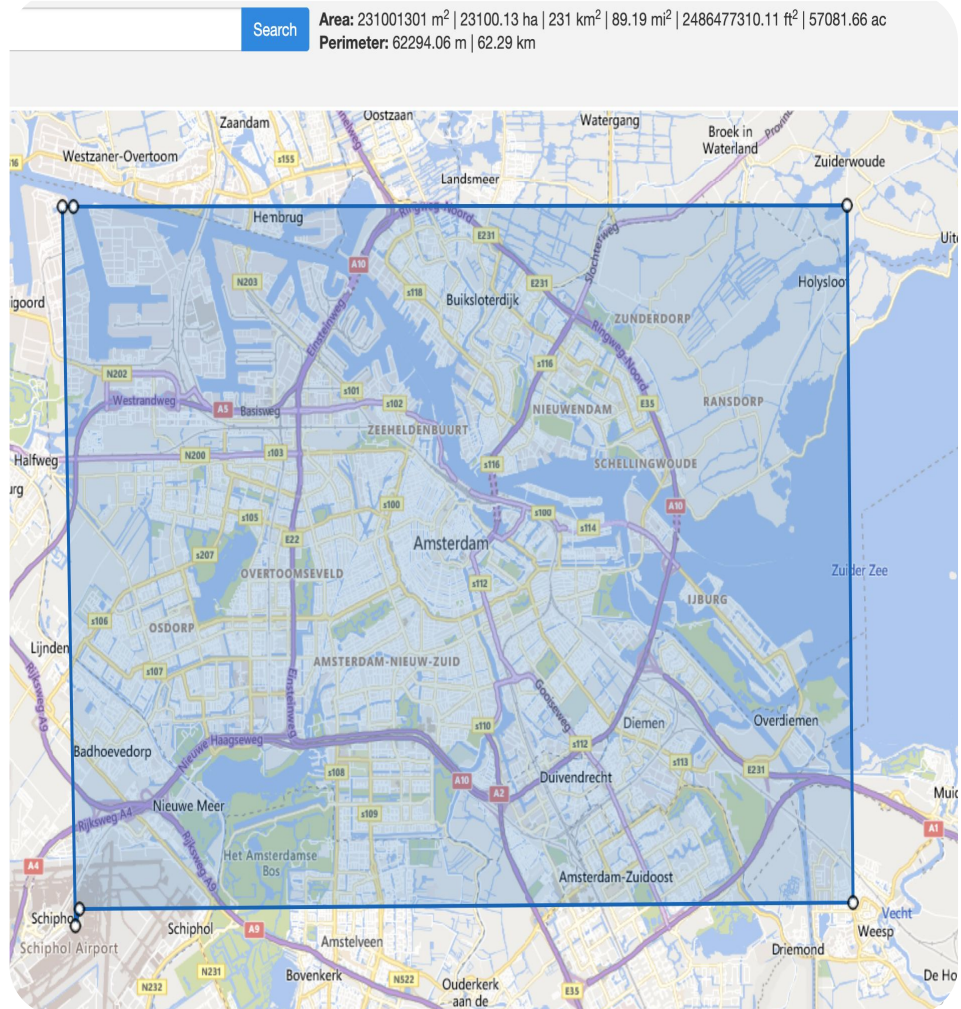
> 200.000 people evacuated

> 18.000 homes destroyed

> 57.000 acres of land burned

While the cause of this wildfire is still under investigation, 60% of the most destructive fires in California have been caused by **defective power lines**

2018 Camp Fire → 153.000 acres



January 2025 Southern California wildfires

29 deaths

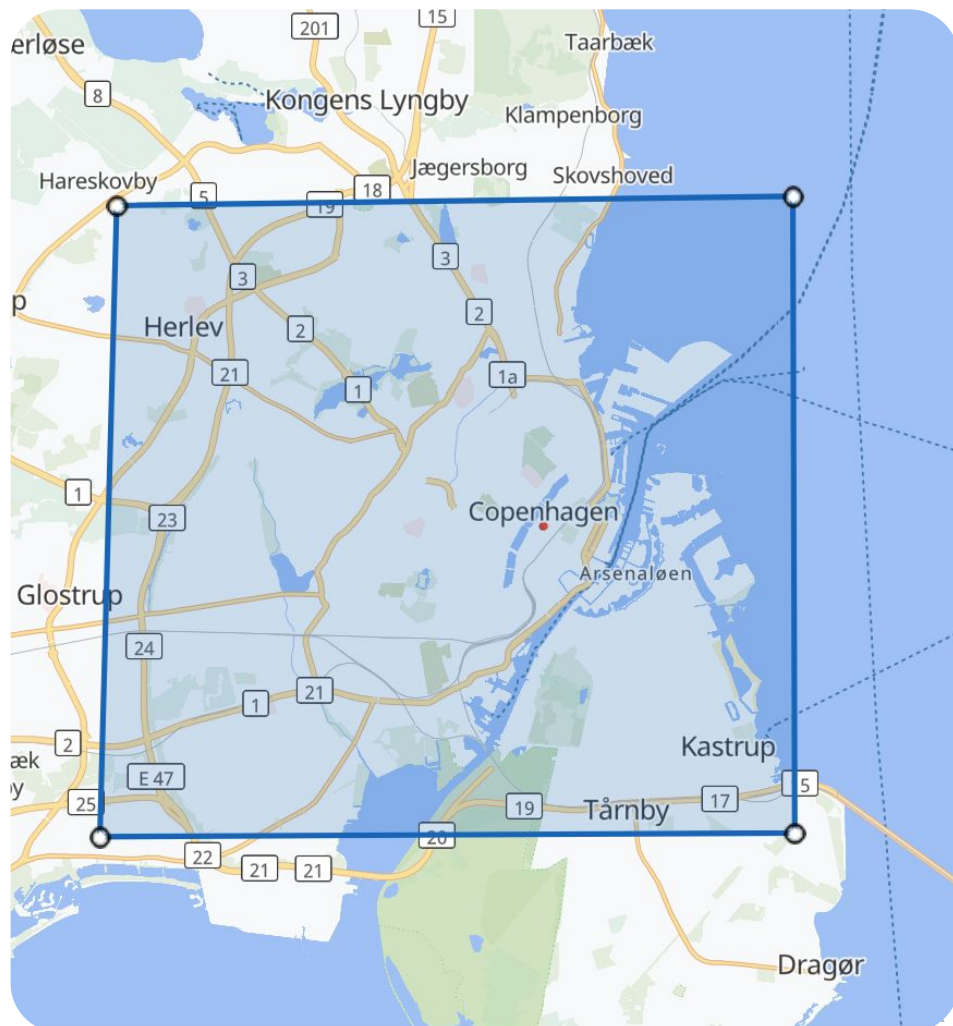
> 200.000 people evacuated

> 18.000 homes destroyed

> 57.000 acres of land burned

While the cause of this wildfire is still under investigation, 60% of the most destructive fires in California have been caused by **defective power lines**

2018 Camp Fire → 153.000 acres



Wildfire prevention is not an easy task...



Power lines are a complex and critical part of our infrastructure

They bring electricity to our communities 365 days a year



Failures are sudden and catastrophic

~ 1350 flights disrupted
~ 300.000 passengers affected

Heathrow Airport shuts down for 24h



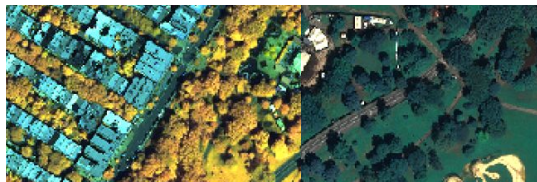
Subject to atmospheric agents

They can get damaged by external factors and inspecting them for maintenance is not easy



Very high resolution (VHR) satellite imagery and machine learning give us actionable insights on tree health, grow-in and fall-in risk, tree species, wildfire risk, and more.

How we built vegetation intelligence



Vegetation data is created from remote sensing sources

Tree detection, tree height, tree health, and tree species from satellite/aerial images



Then combined with relevant data from utility companies

Pole and line location, wildfire maps, trim specifications, terrain and slope maps



And turned into actionable trim recommendations

Cycle prioritization, hazard trees, encroachment, reliability forecasts, contractor audits

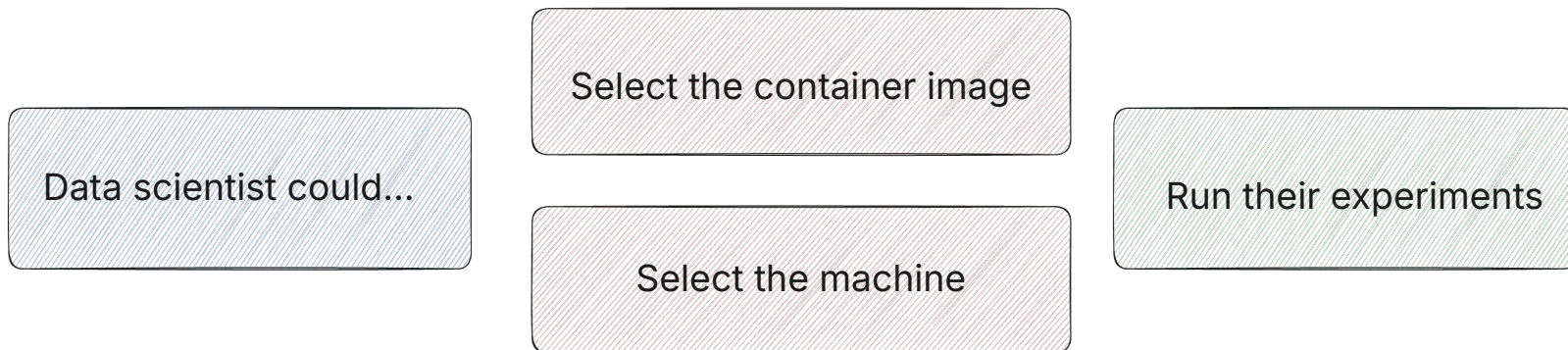
How we got started

How we got started



Move fast, break things

Everything was a **Jupyter Notebook**



Issues with this approach

Repeatability and tracking was **a nightmare**

Does anyone have a local copy of ***delivery_24_0_34_tmp4.ipynb*** ?

Which **machine** did we use for *delivery_78_9_77_test6_final.ipynb* ?

How remembers the **version of pandas** we used in Q1-2021 ?



Very manual
delivery process



Time to automate...

No more Untitled27.ipynb



Two main objectives

Get rid of Jupyter Notebooks

Difficult to maintain
Extremely difficult to test
Tracking with git was 🤬
...

Automate workflow runs

Need for a Data/Workflow orchestrator

Dagster won

Open-source

Fast development

Testable locally

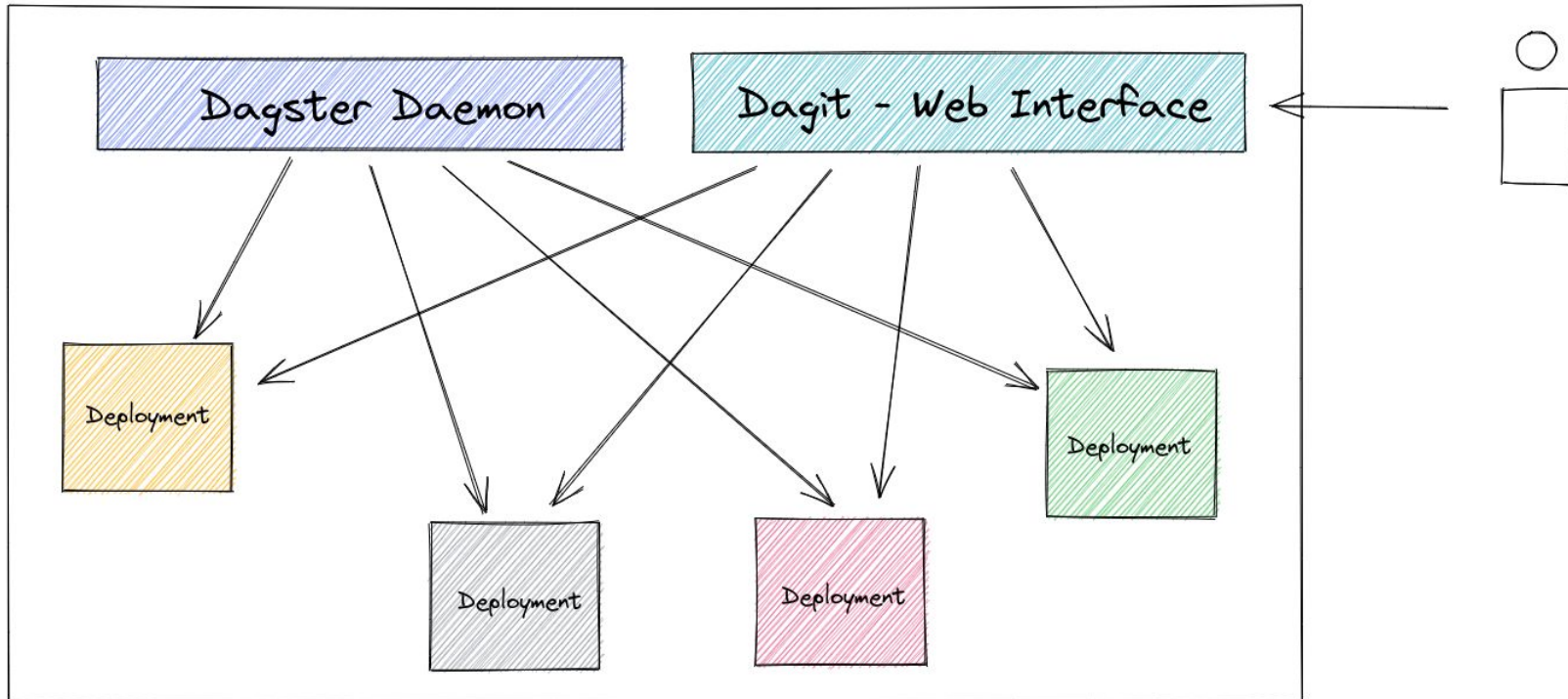
Python modules

Cloud Native

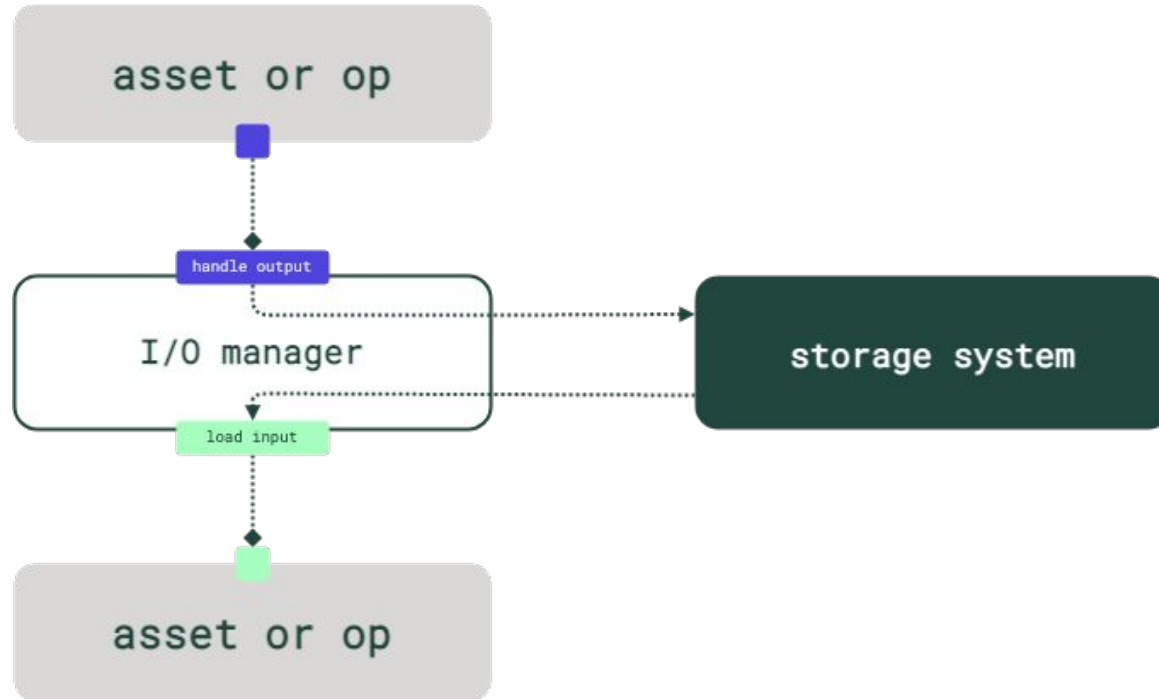


Current Deployment

GKE - Kubernetes

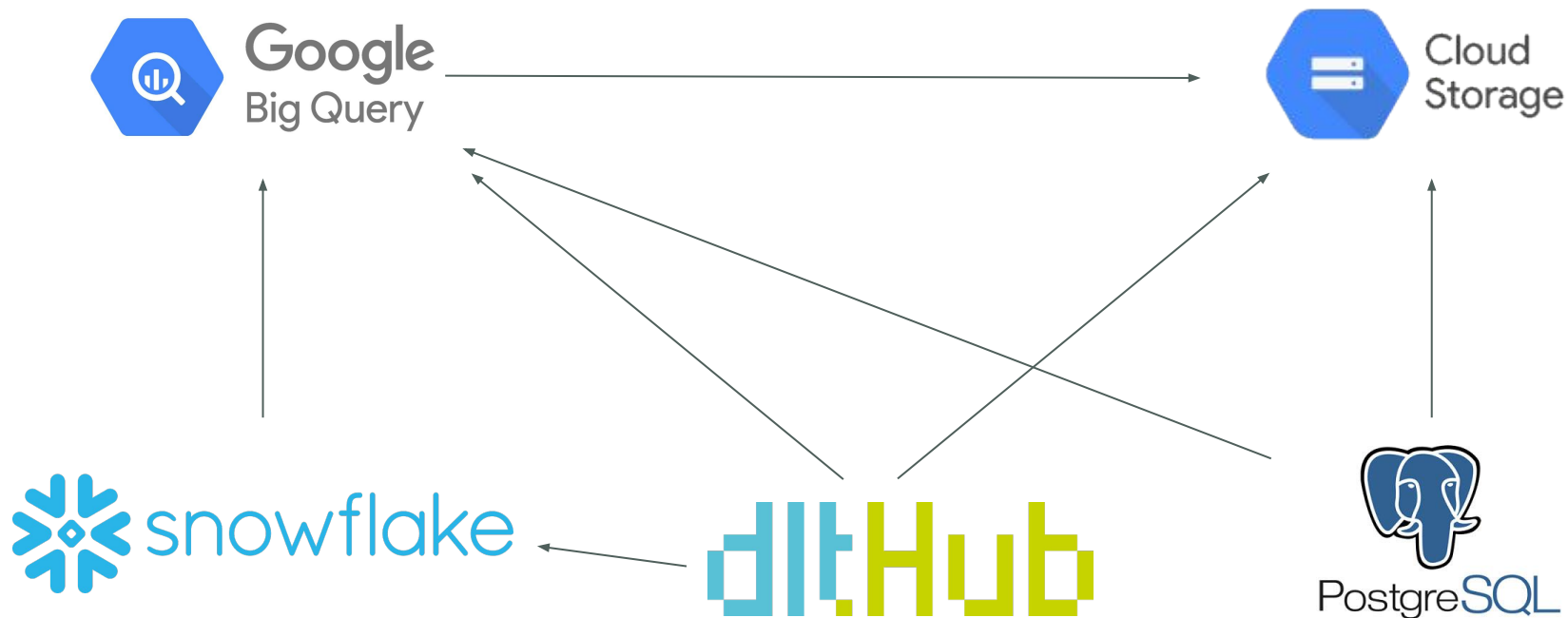


I/O managers

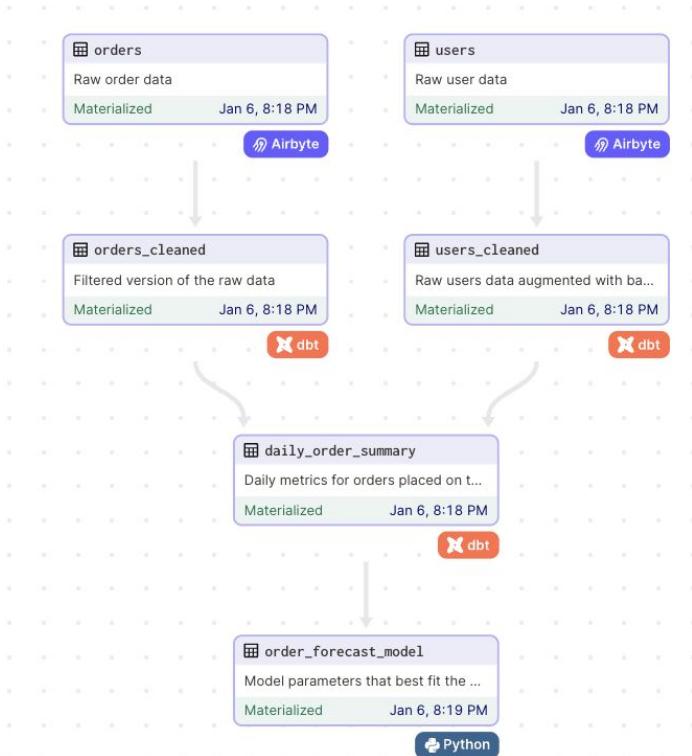


Data tools puts a lot of effort in describing how data is built...

... but often what we care about, is **how data relates to each other** 🤔



Enter the world of Assets...



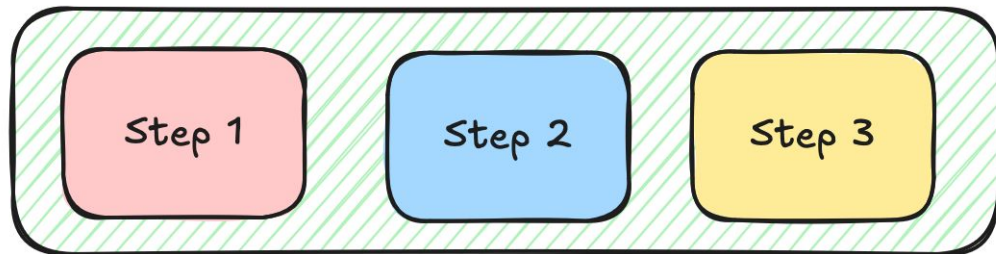
```
@maple_op(  
    include_requests=True,  
    include_requests_cpu="32",  
    include_requests_mem="390Gi"  
)  
def pole_correction_inference_op():  
    # My code down here  
    [...]
```

**Developers can specify the resources
they need to run their pipelines**

In the next iteration of **maple** we plan to
improve its API and avoid any
Kubernetes-specific configuration

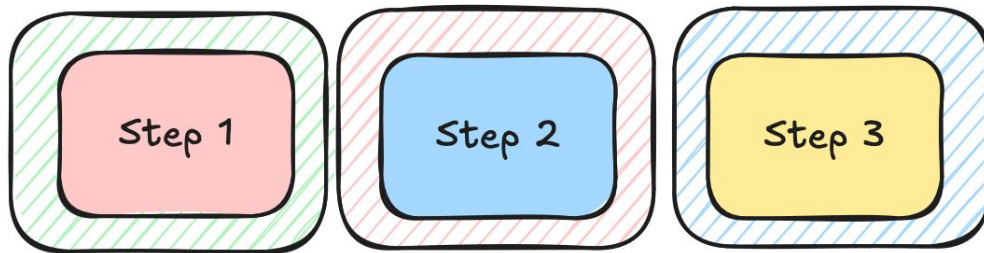
CPU, memory, GPU, scratch disk, node selector, NFS mount, etc...

Pipeline - Single-Pod Single-Node



4Cpus - 8Gb

Pipeline - Multiple-Pods Multiple-Nodes



4Cpus - 8Gb

16Cpus - 32Gb
GPU-enabled

8Cpus - 16Gb
Eph. Storage

The next steps...

Building a platform



Integrations

Pipelines were working fine, but the whole process is **long** and requires **complex logic**

In particular we struggled with:



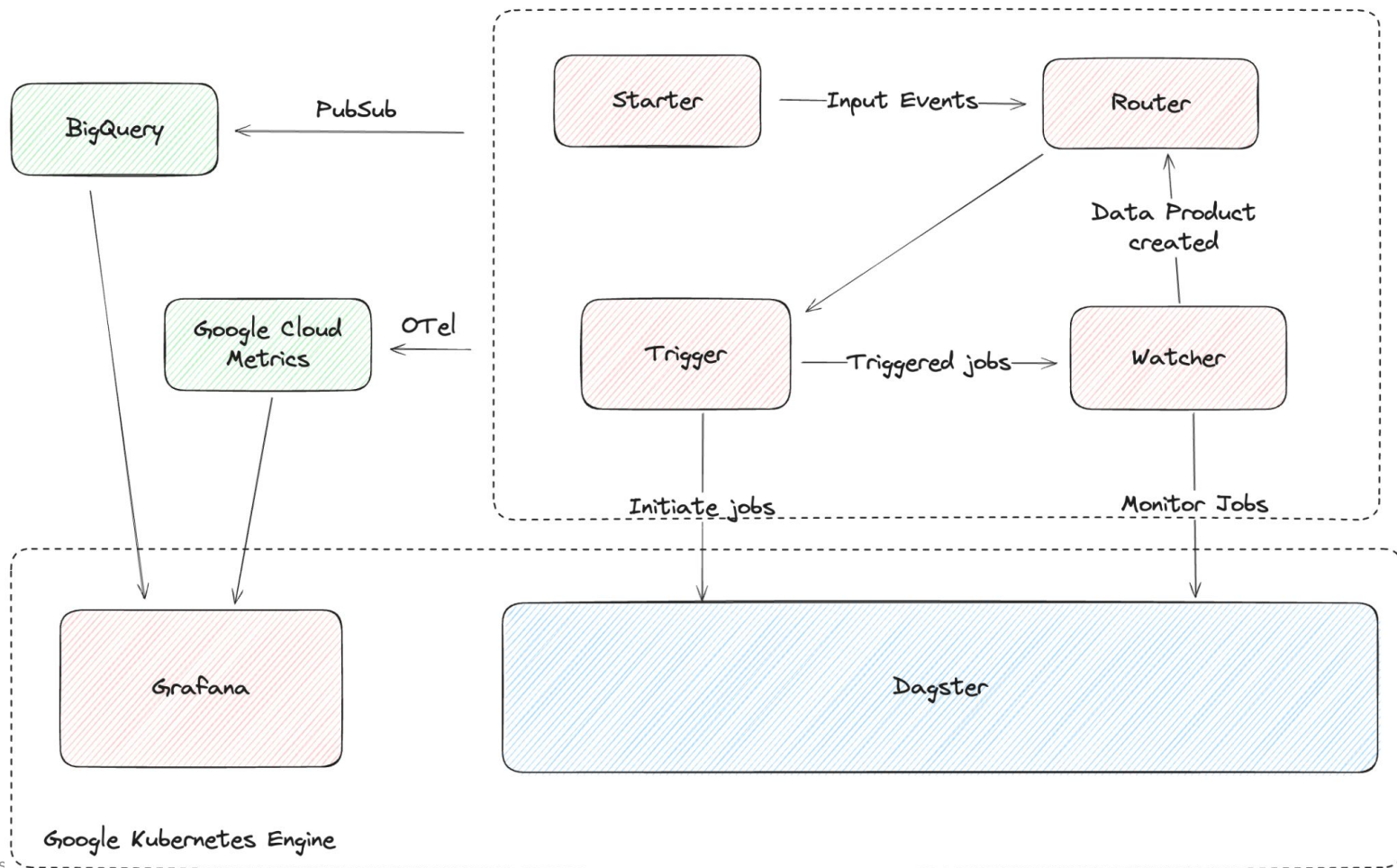
Observability

- Dagster's user interface is nice
- Aggregate statistics per team are difficult to extract

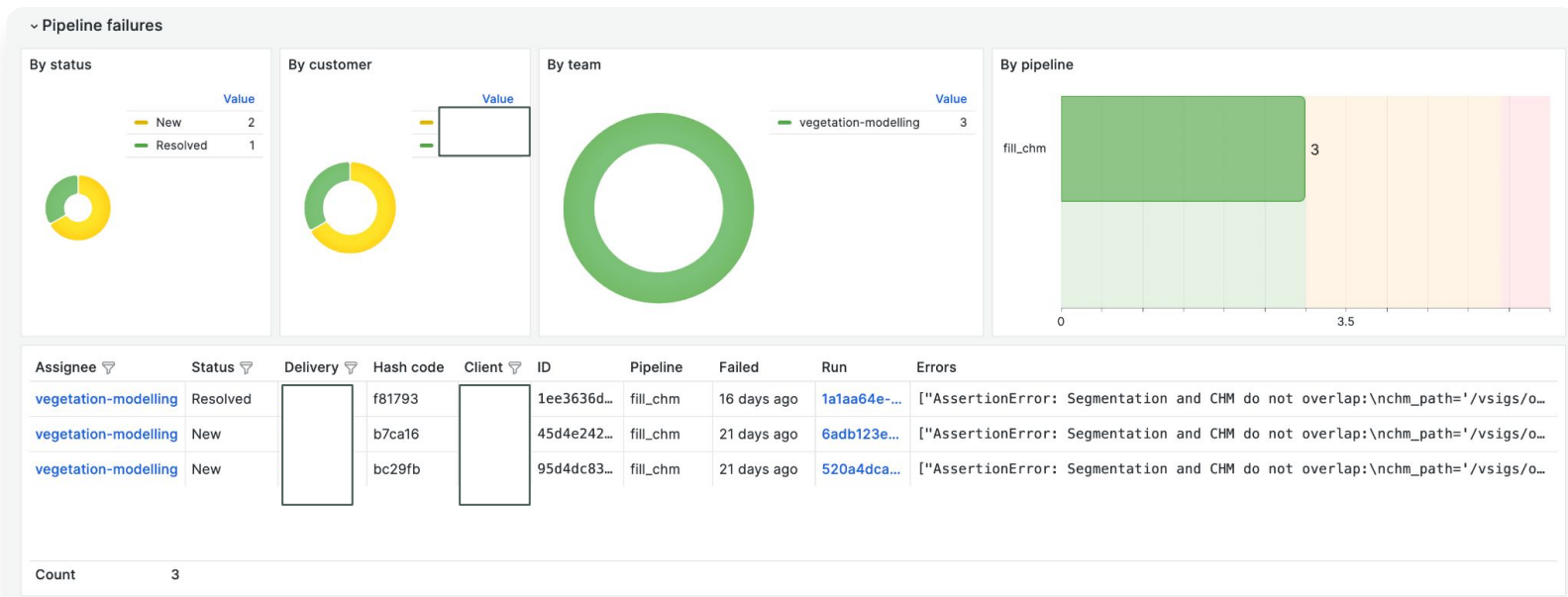


Complex dependencies hierarchies and some retry logics can still be difficult to model in native Dagster

Cloud Run - Communication via PubSub



Orchestration platform dashboards



Connecting all the pieces

#← Also sent to the channel



[redacted] Today at 12:06 AM

Just re-ran this e2e for [redacted] by publishing the `DeliveryCreated` event.

Zero-touch, 30mins to **re-do everything**, no failures (except for risk matrix which will be addressed by the backfill)

This is how all re-runs should be 🧘





A few days ago, we ran
~27k jobs/day with a success rate of 98.3%

What is next

👐 Deliver results to our customers **as hands-off as possible**

- Take the platform and Dagster automation to the next level
- Build on newer Dagster features to make pipelines more reliable to changes

🔗 Break the **one-step-one-node / one-job-one-node** assumption

- Multiple jobs / steps per node
- GKE Node auto-provisioning 🤔

🔥 Avoid (as much as possible) **under-provisioning / over-provisioning** resources

- Maple API should be more straightforward to use
- Improvements on monitoring and observability

What is next

👐 Deliver results to our customers **as hands-off as possible**

- Take the platform and Dagster automation to the next level
- Build on newer Dagster features to make pipelines more reliable to changes

🔥 Avoid (as much as possible) **under-provisioning** / **over-provisioning** resources

- Maple API should be more straightforward to use
- Improvements on monitoring and observability



Thank you.



Andrea Giardini

andreagiardini.com